Replication Study: Interpretable eDNA Species Classification

Reproducing and analyzing the paper *Creating interpretable deep learning models to identify species using environmental DNA sequences*

Abdullah Al Fahad

INTRODUCTION TO BIOINFORMATICS: 23072
IUI

20 April, 2026

Motivation

Environmental DNA (eDNA) workflow

- eDNA enables non-invasive biodiversity monitoring from water or soil.
- The classification task is: **given a DNA sequence, predict the species label**.
- This is valuable for ecological surveys, habitat monitoring, and large-scale species detection.

Why machine learning?

- eDNA studies can produce very large numbers of reads.
- Prior CNN work showed fast sequence classification, but with black-box predictions.
- **Can we keep strong accuracy and make the model interpretable?**

Challenges

Prediction challenge

- Identify fish species directly from **short eDNA sequences** ($\approx$ 64 bases).
- Handle **noisy**, **ambiguous**, and **variable-quality** reads.
- Preserve enough **performance** to be useful in practice.

Interpretability challenge

- Standard CNNs can classify well but do not explain **why** a species was predicted.
- **Post-hoc saliency** is not always faithful to the model's actual reasoning.
- The paper proposes a **prototype-based model** so decisions can be linked to **human-readable subsequences**.

Dataset

Data source

- 12S fish eDNA sequences from **French Guiana**
- Samples collected from the **Maroni** and **Oyapock** rivers
- Data came from **200 collection sites** between **2014 and 2020**
- Initial dataset covered **368 freshwater fish species**

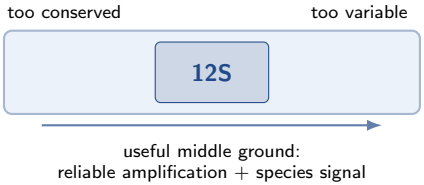
Sequence characteristics

- Marker region: **12S rRNA**
- Average read length: **64 bases**
- Short reads are typical in eDNA workflows
- The task is multiclass species classification

Why the 12S rRNA region?

Why this marker is useful for eDNA

- The **12S ribosomal RNA region** is widely used in vertebrate and fish eDNA studies.
- It is **conserved enough** to be amplified reliably from short environmental DNA fragments.
- It is also **variable enough** to preserve species-level signal.
- This makes it a practical compromise for **biodiversity monitoring** and species classification.



Preprocessing and input representation

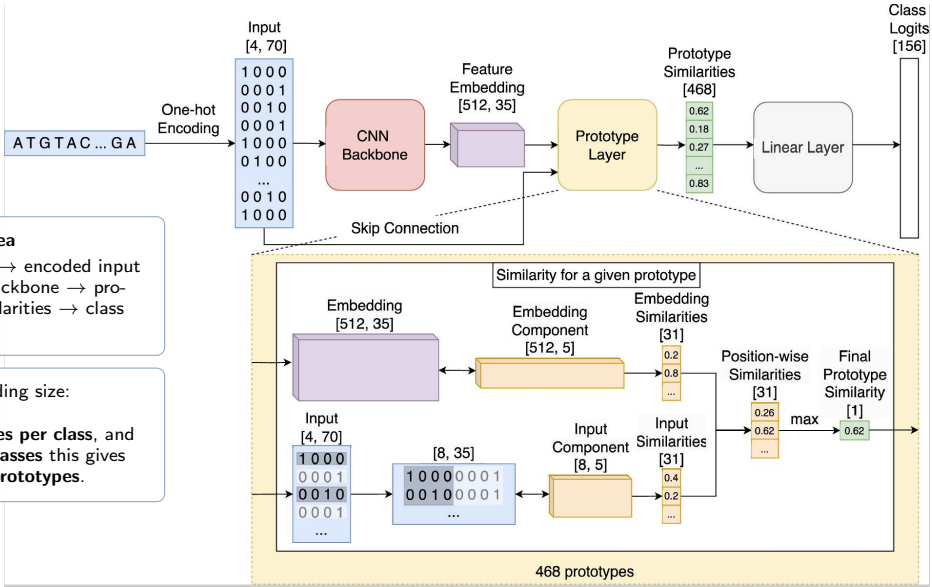
Preprocessing

- Remove tag, primer, and reverse-complement artifacts
- Remove species with fewer than **2 sequences**
- Final modeling setup keeps **156 species/classes**
- Oversample train/validation data; keep the **test set unbalanced**
- Online augmentation during training:
  - insert 0–2 random bases
  - delete 0–2 random bases
  - mutate each base with **5%** probability

Input representation

- Pad or truncate each sequence to length **70**
- Encode DNA as a **4-channel 1D input**
- A/T/C/G use one-hot encoding
- Ambiguous IUPAC symbols use probability vectors

Methodology: from DNA read to species prediction



Pipeline idea

DNA read → encoded input → CNN backbone → prototype similarities → class logits

the embedding size: [512, 35]
3 prototypes per class, and with **156 classes** this gives **468 total prototypes**.

Prototype layer and skip connection

What is a prototype?

Prototype = a learned reference pattern in feature space.

How it is used:

$$\text{input patches} \longrightarrow \text{compare with prototype} \longrightarrow \text{keep strongest match}$$

Meaning: the maximum similarity becomes evidence for the class prediction.

What does the skip connection do?

Role: it combines latent CNN features with information closer to the original input.

Why it helps:

- keeps prototype matches tied to recognizable subsequences
- reduces reliance on only abstract hidden features
- makes explanations easier to trace back to the DNA read

Training ProtoPNet

Warm-up
initial prototype learning
→
Joint training
backbone + prototypes
→
Prototype push
nearest training patch
→
Evaluation
test + analysis

Training flow

First train the model to learn useful latent features and prototype representations, then **push** each prototype toward a real training patch, and finally evaluate both classification performance and interpretability outputs.

Replication setup

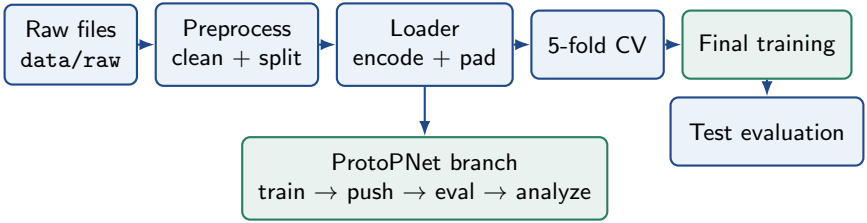
What I reproduced

- **Encoding:** IUPAC-aware 4-channel DNA input
- **Input length:** pad/truncate to 70
- **Augmentation:** insert / delete / mutate
- **Task:** multiclass species prediction
- **Training:** stratified cross-validation

What I adapted

- **Data handling:** small dataset with **156** species, **1111** reads and **7.11** average read per species
- **Backbone:** simplified `Small_Best_Updated_like`
- **Environment:** CPU-based run
- **Extra study:** backbone vs. ProtoPNet comparison

Replicated pipeline



Artifacts produced

Processed splits, CV metrics, training logs, checkpoints, result summaries, confusion matrices, and ProtoPNet interpretability outputs.

Original paper vs my replication

Method and data comparison

Aspect	Original paper	My replication	Match
Task	Species classification from eDNA reads	Same task	**Close match**
Encoding	4-channel DNA encoding with IUPAC handling	Same idea	**Close match**
Input length	Fixed length of 70	Same: 70	**Close match**
Augmentation	Insert / delete / mutate during training	Same style online augmentation	**Close match**
Backbone model	Shallow 1D CNN	Simplified `Small_Best_Updated_like`	**Partial match**
ProtoPNet pipeline	Prototype learning + push + interpretation	Train + push + eval + analysis	**Partial match**
Data setup	Original study dataset and split protocol	Repository/local processed split: 780 / 156 / 175	**Adapted**

Experimental setup

Data

- `seq` → `species_cat`
- Train / val / test: 780 / 156 / 175
- 156 classes
- Cleaned, label-encoded, and standardized before training

Training

- Adam + cross-entropy
- 5-fold stratified CV with early stopping
- Final training: 14 epochs on CPU
- Augmentation: insert / delete / mutate

Original paper results

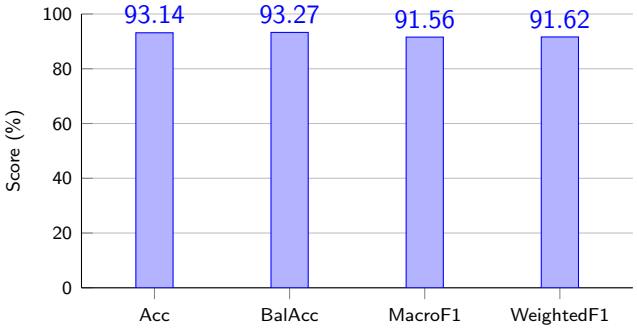
Main findings

- **1-layer CNN** gave the best accuracy / interpretability trade-off
- **Noise augmentation** improved robustness
- **Short prototypes** worked best ($\approx$ 3–5)
- **Latent weight** around 0.7 balanced accuracy and interpretability

Key numbers

- Updated CNN: **94.48%** accuracy
- ProtoPNet: **95.31%** at $\kappa = 0.7$
- ProtoPNet: **95.66%** at $\kappa = 1$
- For latent weights > 0.5, accuracy stayed within about **1%**

Replicated results: backbone CNN replication



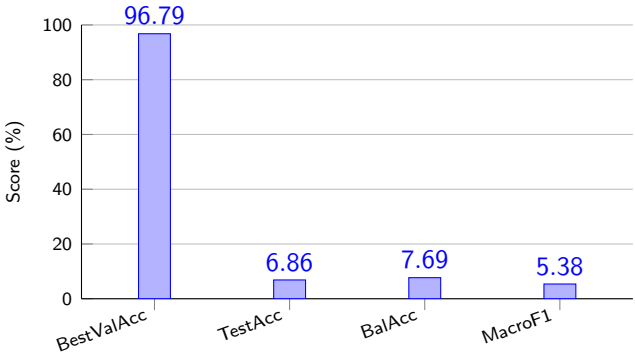
Backbone outcome

- accuracy = **0.9314**
- balanced accuracy = **0.9327**
- macro precision = **0.9071**
- macro recall = **0.9327**
- macro F1 = **0.9156**
- AUC OVR macro = **0.9993**

Interpretation

The core CNN replication worked well on my local 156-class split. This means the backbone pipeline itself is sound and provides a strong baseline for judging the ProtoPNet extension.

Replicated results: ProtoPNet



ProtoPNet outcome

- best validation accuracy = **0.9679**
- test accuracy = **0.0686**
- balanced accuracy = **0.0769**
- macro F1 = **0.0538**
- weighted F1 = **0.0480**
- AUC OVR macro = **0.9947**

Interpretation

The full ProtoPNet pipeline ran successfully and produced interpretability artifacts, but after push the top-1 classifier failed badly on the test set even though ranking quality (AUC) stayed very high.

Sources of Performance Difference

Most likely

- My replication setting is difficult: **156 classes**, only **175 test samples**
- ProtoPNet is likely sensitive to push-stage and tuning choices
- The backbone remained strong, so the issue is more about ProtoPNet stability than task impossibility

Main claim

The gap is best explained by tuning sensitivity and replication difficulty, not by failure of the core ProtoPNet idea.

How Prototype Helps

Why is

CCCCTCTAAACAACCCATAACTATACTTAAACATCA
CACACGTATACCAGTAGGAGAGACAAGTCGTANN

classified as a *Gymnotus anguillaris* (a kind of zebra knifefish)?

	Matched subsequence	Prototype	Similarity score		Class connection		Points contributed
Evidence for *Gymnotus anguillaris*:							
a	...CAGTAGGAGAGACAAG...	TAGGAGAGAC	1	×	1.93	=	1.93
b	...ACCAGTAGGAGAGACA...	AGTAGGAGAG	1	×	2.08	=	2.08
c	...GAGAGACAAGTCGTAN...	AGACAAGTCG	0.94	×	1.66	=	1.56
						+	
		Total points to *Gymnotus anguillaris*:					5.57
Evidence for *Japigny kirschbaum*:							
d	...GTAGGAGAGACAAGTC...	GGAGAGGCAA	0.58	×	1.84		1.07
e	...TACTTAAACATCACAC...	TTAAACCAGG	0.57	×	2.16		1.23
f	...AACATCACACACGTAT...	ATCACGCTAT	0.53	×	2.26		1.20
						+	
		Total points to *Japigny kirschbaum*:					3.50

Lessons from the replication

Technical lessons

- **ProtoPNet is sensitive:** performance depends strongly on prototype length, latent weight, and push behavior.
- **Preprocessing matters:** encoding, cleaning, and augmentation are part of the method, not minor details.
- **Simple models can work well:** the shallow backbone was already very strong for this task.

Scientific lessons

- **Interpretability must stay stable:** explanations are useful only if prediction quality remains reliable.
- **Replication is not just rerunning code:** it tests whether the original claims still hold in a new setup.
- **Multiple metrics are necessary:** different metrics exposed different failure modes.

Conclusion

- The **shallow CNN** baseline transferred well.
- The ProtoPNet pipeline ran end to end and produced explanations.
- The main gap was performance stability, not pipeline functionality.

Closing message

This replication supports the main idea of the paper, while showing that interpretable prototype models are more fragile in a small many-class setting.

References I

Samuel Waggoner, Jon Donnelly, Rose Gurung, Laura Jackson, and Chaofan Chen.
Creating interpretable deep learning models to identify species using environmental DNA sequences.
Scientific Reports, 2025.